

Evaluating IPAD under generator shift

Preliminary protocol and score-evaluation software

Hongxi Pu | Working draft for author review | 3 October 2026

Abstract

This work-in-progress document proposes a focused evaluation of an inverse-prompt text detector under a change in the text-generating model. It specifies a held-out design, score provenance, fixed decision rules and uncertainty reporting. A companion Python tool evaluates saved scores and has been checked using artificial numerical fixtures and an independent metric implementation. No IPAD inference or new text-generation experiment has been performed for this report. Consequently, it contains no empirical detection results. The immediate research objective is to establish a reproducible evaluation path before assessing whether performance transfers to a generator outside the original evaluation.

Research question and scope

For a fixed IPAD configuration and decision threshold, how do detection performance and human-text false-positive rates change when AI texts are produced by a previously unevaluated generator, while domain and source prompts are controlled?

The first experiment will address generator shift in one domain. Domain transfer and paraphrasing are later extensions, not additional completed studies. An unseen generator must be checked against the full original paper and released evaluation data before that description is used. No generator, corpus or checkpoint combination has yet been finalized for the new experiment.

Relationship to prior work

IPAD provides the prior methodological foundation [1]. The proposed study evaluates transfer of a frozen detector rather than asserting a new detection method or improvements over published results. It separates detection accuracy from the usefulness of reconstructed prompts: plausible explanations alone do not establish authorship or faithful reasoning.

Current completion status

Completed on 3 October 2026	Not yet completed
Protocol draft and inspection of public model resources.	Final data/model choices and complete inference reproduction.
Saved-score evaluator; 12 unit tests; 200 numerical cross-check cases.	Real predictions, new performance findings, public repository or DOI.

This is a protocol and software draft, not a peer-reviewed publication or a completed empirical technical report. Dates refer to this version of the work.

Proposed experimental design

Sample construction

A manageable pilot would use 150 source groups from one licensed, provenance-documented corpus. Each group contains a source prompt and an independently sourced human answer. Randomly assign 50 groups to source validation and 100 to held-out testing before generating text. Keep all descendants of a prompt or source document in the same split. Check normalized exact duplicates and near duplicates separately; group IDs alone do not prove absence of contamination.

Planned condition	Human / AI records	Purpose
Source validation: 50 groups	50 / 50	Verify and, if prespecified, calibrate source-only settings.
Reference-generator test: 100 groups	100 / 100	Measure the frozen reference configuration.
Shifted-generator test: same 100 groups	100 / 100	Change the generator while retaining matched prompts and human controls.

This design has 500 evaluation rows but 400 distinct text instances if human test controls are reused across conditions. Preserve shared group IDs. Report each condition separately; do not pool repeated controls as independent observations. These counts are a proposed pilot size, not a completed collection or a statistical power guarantee.

Freeze configuration before examining target scores

Record exact generator identifiers and versions, sampling settings, generation date, corpus revision, source IDs and text hashes. Use a single regeneration configuration for both test conditions. Pin the base model, adapters, tokenizer, instruction templates, numeric precision, maximum lengths and score aggregation. Select the threshold from a documented original setting or source validation only; record the choice and rationale before viewing shifted-test predictions.

Scoring and error handling

Use continuous scores with a verified direction: higher means more likely AI-generated. Preserve component scores, reconstructed prompts and regeneration outputs alongside sample IDs. Confirm normalization and yes/no token handling instead of treating a single vocabulary probability as a calibrated binary probability. Keep a manifest of attempted samples and all inference failures. Resolve failures or explicitly report coverage; do not silently drop difficult examples.

Interpretability observations

Prespecify a small sample of source groups for qualitative inspection, covering both test conditions. Inspect reconstruction relevance and consistency evidence without presenting examples as quantitative proof of explanation faithfulness. Do not select only persuasive examples after observing detector errors.

Evaluation software and verification

Implemented functions

The companion evaluator consumes a CSV containing `sample_id`, `group_id`, `split`, `condition`, `label` and `score`. It reports accuracy, precision, recall, F1, human-text false-positive rate and tie-aware AUROC for each held-out condition. Prediction uses score greater than or equal to an explicitly supplied threshold. Undefined metrics remain null rather than being silently replaced with zero.

For uncertainty, the tool resamples source groups, retaining the rows within each group, and produces 95% percentile intervals with a fixed seed. It reports undefined bootstrap replicates and withholds intervals if too few replicates are usable. Separate intervals are not a significance test for the difference between matched conditions. Paired difference intervals are not implemented in this version.

Validation actually performed

Check	Executed outcome
Unit tests	12 passed: ranking, ties, threshold boundary, undefined metrics, duplicate IDs, split leakage, invalid scores, bootstrap repeatability and pipeline metadata.
Independent metric comparison	200 artificial cases agreed with scikit-learn 1.8.0 for AUROC, accuracy and F1 within absolute tolerance 1e-12.
Example pipeline	Executed on 8 hand-constructed numerical rows with 2,000 group-bootstrap draws; output explicitly labeled software-validation.

These checks establish limited software correctness. They do not measure IPAD, establish robustness, validate an inference pipeline or supply evidence of scientific novelty. The artificial fixture contains no actual human or model-generated text. Its numerical metric values are deliberately not presented as a research-results table.

Reproduction commands

```
python3 -m unittest discover -s tests -v
```

```
python3 evaluate.py --predictions examples/fixture_predictions.csv  
--metadata examples/fixture_metadata.json --threshold 0.5  
--purpose software-validation --bootstrap 2000  
--out validation/fixture_metrics.json
```

Run the second command as one shell line or use the continuations in README.md. The example threshold 0.5 is only a software-test setting. Real evaluation requires completed study metadata and genuine prediction scores. The scorer itself uses only Python 3.10+ standard-library modules; the report generator additionally needs ReportLab.

Limits of automated checking

The tool checks supplied identifiers and the declared expected row count. It cannot verify that a corpus is genuinely human-authored, that metadata is accurate, or that different IDs do not hide duplicate text. Corpus provenance, training overlap and the scientific meaning of scores require separate review.

Reproduction path and next steps

Available resources and unresolved integration

The IPAD model repository contains adapters and test resources [2], linked from the code repository [3]. The base model is Phi-3-medium-128k-instruct [4]. This session had approximately 9.7 GiB host memory and no available NVIDIA GPU utility; ordinary full-precision loading was not feasible. Quantized or offloaded inference was not attempted.

Source inspection also identified integration questions: usage comments and the paper appear to associate the PTCV/RC names differently; the two released helper filenames do not clearly match their contents; and probability extraction depends on a local engine-generated file. Confirm actual training formats and checkpoint semantics before reproducing the complete pipeline. These are unresolved implementation questions, not evidence against the published detection results.

Minimum next research steps

1. Review the protocol and choose one domain and one genuinely new generator.
2. Confirm the exact adapter/module mapping and reproduce a small known input through the full pipeline, including regeneration and score fusion.
3. Freeze source groups, generation parameters and the threshold before target evaluation.
4. Collect real predictions and retain failure logs and provenance.
5. Run the evaluator, investigate errors, and write findings only from the resulting records.

No collaboration commitments, funding, model access or experimental outcomes are asserted in this draft. An initial public release, if chosen after review, should be labeled an evaluation protocol and software work in progress. A later report can add empirical findings with its actual run dates and version history.

References and pinned resources

[1] Chen et al. IPAD. NeurIPS 2025. Official paper:
[NeurIPS proceedings record](#).

[2] huggingface.co/bellaafc/IPAD
Inspected revision: e57952ab8f36a77421a02f96ac2a2fa13d324b6e.

[3] github.com/Bellaafc/IPAD-Inver-Prompt-for-AI-Detection
Public README and file tree inspected on 3 October 2026.

[4] huggingface.co/microsoft/Phi-3-medium-128k-instruct
Inspected revision: a088b37c71d441ab6d862bb3fcfe6165b3014702.

AI assistance was used for drafting and coding. Scientific choices, source alignment, authorship and any public release require researcher review. The package includes no Meta data or code.